

Creating the homeserver for messenger access on Windows Phone 10



Requirements:

- Some form of Linux machine/Virtual Machine
 - I have Debian running on my server so I will write this guide assuming you are also using Debian (although setup for similar distros shouldn't differ all that much)
 - I don't see why WSL couldn't work but I don't intend on testing it
- Admin access to the router your machine connects to
- A registered domain (we will get one with DuckDNS for free)
- Lots of patience

The setup briefly:

Our Debian machine acts as a Matrix homeserver, for this we will use Synapse. At this point by itself the server can be used for many things, Matrix's goal is to provide decentralized, secure online communication for everyone. At this stage, if proper access is provided you can log in to your server with a Matrix client like Element.

So to make it function as we want it to we will use the mautrix-meta bridge. This is essentially a puppet to our Facebook account. So what it does is it creates a bot user in our synapse server, after providing your messenger.com login token it can access messenger "legally".

The way it works is for every chat you have in messenger: marketplace, group, 1 on 1 etc. It creates an invite. Inside the Element app (which we will use as the generic Matrix client) you will accept every chat you would like to see on your Windows Phone. Then the bot downloads the configured amount of chat history to the server so it displays properly in the chat window. Sending messages is more fun, if you do so you send it only to your server. The bot sees what chat room it goes to and, in your place, sends that message to the Meta servers. On Meta's side its as if you logged onto the official messenger app or website and sent a message.

And after this is configured we can get the Windows Phone Matrix app and connect to our server.

Disclaimers:

- In our bridge config we can set a backlogging parameter which essentially says how much chat history should be downloaded + the time interval for downloads. Not all of you chat history will show up at once because polling the Meta servers a lot in very short time intervals could result in your account getting banned

- I am not responsible for any negative outcome this may have on anyone, your meta account etc.
- But of course I will very happily provide support to anyone that needs it :)

Setting up Debian

If you already have a Debian server you can skip this part. If its a VM make sure its network adapter is either bridged or you configured NAT for it.

For this section of the guide I will be using VirtualBox for the VM, but you can use any preferred hypervisor.

0. Enabling VM support on your computer

If you are running Windows/Linux you most likely have an option in your motherboard BIOS to enable virtualization support (VT-x, AMD-v). Usually you get in by pressing DEL, F2 or F12 after powering on your computer, but its the best to search online how to do so, for your specific machine or motherboard online.

If you are running windows you shouldn't need to modify anything, same goes for MacOS as far as Im aware.

1. Install VirtualBox

Visit <https://www.virtualbox.org/wiki/Downloads> and download the one appropriate for your system. Go through the setup normally, there is no need to change anything.

2. Download the Debian ISO

Go to <https://www.debian.org/distrib/> and I recommend downloading the complete installation image, so "64-bit PC DVD-1 iso"

3. Create the virtual machine

Inside VirtualBox click on New, then name your machine and specify where the ISO you downloaded is. Uncheck "Proceed with Unattended Installation"

Then click on Specify virtual hardware. A rule of thumb for virtual machines is never give more CPU cores or memory than half of what you have. This can always be changed later so if you can give it two CPU's and 4GB (4096MB) of RAM. But while we are messing around setting it up feel free to give it more, as much as you can so our UI and everything goes as smooth as possible. Afterwards when you just boot it up for it to run in the background you can give it less.

Then click on Specify virtual hard disk. Just to be safe Id give the machine minimum 50GB. Dont be afraid to give it more, unless you check 'Pre-allocate Full Size' it will only occupy as much space as it actually occupies.

And click on Finish.

Then go into the settings of the machine and navigate to the Network section. By default Adapter 1 is enabled and set to NAT. This means that the server isnt exposed to the network outside of your computer, instead it communicates through the address of your computer.

If you want to you can change NAT to Bridged mode. This will make the server connect "directly" to the router your computer is connected to, so it will receive an IP address from it.

If you keep it on NAT mode that means we have to configure Port Forwarding in Vbox so the server can be reached from the outside. To do so click on Port Forwarding and add a new rule with the add button in the top right.

Name it something like “Synapse Inbound”, keep the Protocol on TCP, Host IP blank, Host Port as 8443, Guest IP blank and finally Guest Port as 8443.

And click on OK, then once again OK out of the settings and we can start the machine.

Its a good time to also open up port 8443 in your operating systems firewall. In windows you can do that by opening up the Windows Defender Firewall and selecting advanced settings (or opening Windows Defender Firewall with Advanced Security).

Click on Inbound rules, then add a new rule.

Select a Port rule

Then set it to be a TCP rule and for “specific local ports” type in 8443.

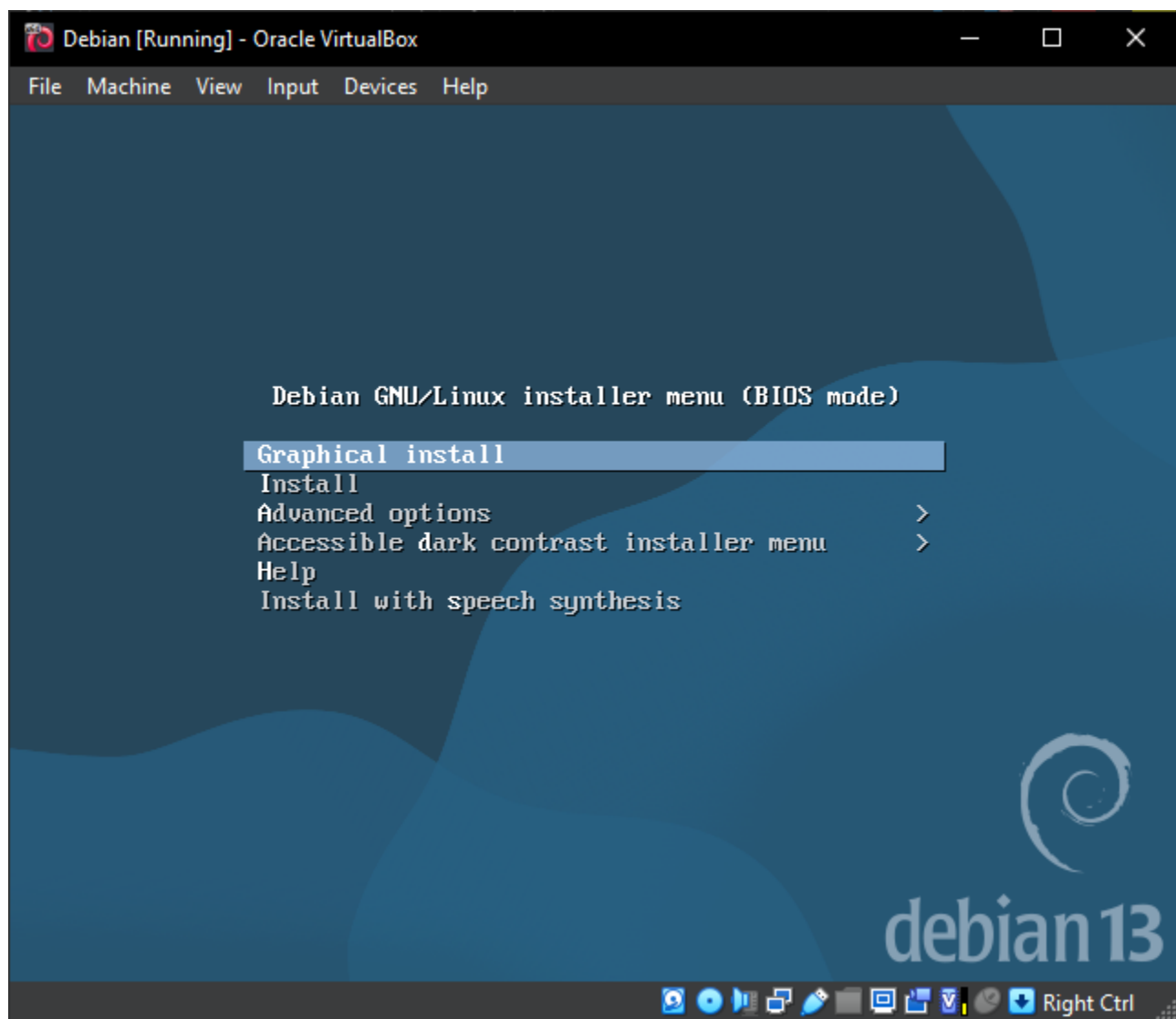
Then leave “Allow the connection” checked.

If your home network connection in windows is set to Private you can just leave Private checked and everything else unchecked. But you can keep all three options checked.

Name the rule something like “Synapse inbound”.

4. Installing Debian

With the Debian VM selected click on Start. It should boot up to a screen with a couple options, select Graphical install



4.1 Then select your install language, location and keyboard layout.

4.2 After loading and configuring itself it should ask for a hostname. Name it whatever you like, I recommend something along the lines of “synapse-server”.

4.3 Then it will ask for a domain name, if you have a running domain name in your local network that's what you should type here, if you don't know about it you probably don't have one configured and that's not an issue. In that case type in something like “home.arpa”.

This means that this server, if this same domain is configured in your router, can be reached locally through “synapse-server.home.arpa”.

4.4 The setup will then ask you for the root user password. I recommend you leave everything blank so that the first user we create will become the root user (If you are unfamiliar with *nix terms, you could say root is the administrator account). Leaving the root account disabled is akin to disabling the Administrator account under windows.

4.5 Next type in the full name of your user, it can be your real name or anything you'd like.

4.6 Then the setup will suggest a username based on the real name, again this can be anything you want it to be.

4.7 And now type in the password you will use to log onto your server.

4.8 Time for the disk partitioning, if you have any preference of course go ahead but if you keep “Guided – use entire disk” selected you will be just fine.

4.9 In the next screen just click continue.

4.10 Then “All files in one partition” is good.

4.11/12 And finally select Finish partitioning and write changes to disk, in the next screen select yes. The system is finally installing!

4.13 At the “Configure the package manager” keep No checked and continue.

4.14 At the next screen for using a network mirror select “yes”.

4.15 Then select your country or the country closest to yours at “Debian archive mirror country”.

4.16 And select whichever you want at “Debian archive mirror”;

4.17 At the HTTP proxy most people will be good with leaving it blank.

4.18 At the “Participate in the package user survey” select whichever you want.

4.19 Next comes “Choose software to install”. Leave standard system utilities selected and we can also select a desktop environment. By default a DE, depending on which, will make the VM consume more resources. Since we only want it to run in the background and consume as little as possible it would make sense to choose none.

But luck has it that we can always disable it and log in to plain CLI so choose whichever you want, especially for someone new to linux it will make life easier.

The DE that consumes the least amount of resources in my experience is “Xfce”, if you also choose this one uncheck “Debian desktop environment”, check “Xfce” and continue.

4.10 At the “Install the GRUB boot loader...” select yes.

4.11 At the “Device for boot loader installation” select the line that starts with /”dev/sda” and continue

4.12 AND finally the system is installed, select continue to reboot.

We will do most, essentially all of our work in the Terminal.

4.13 Now we want to update our server for the first time, but to successfully run the commands we need to edit a file so that it doesn’t try to update from the installation ISO. This will be a good introduction to Nano, the terminal text editor I will be using in this guide.

Open the terminal which you can find by searching for it in the software search bar.

Type in:

```
sudo nano /etc/sources.list
```

Running any command as sudo is akin to running something with administrator rights. Type in your user password and the file will open.

You can navigate the file with the arrow keys, delete the line that starts with “deb cdrom:” and save the file with ctrl+s, then exit Nano with ctrl+x.

Then we can check for updates by running

```
sudo apt update
```

And if it found anything to upgrade

```
sudo apt upgrade
```

To power off the machine you can type in the terminal

```
sudo poweroff
```

And to reboot you can type

```
sudo reboot
```

And the base for our server is good to go!

Network and DynamicDNS

You should be familiar with your routers model as every step involving the router will be unique to it, if you don't know how you will have to search online.

1. Static IP

Set a static local IP address for your computer/VM. This is needed because when we configure port forwarding rules, those will be bound to an IP address. If the router or your computer restarts and you don't get the same IP your server will not be reachable.

So we need to log into the routers web interface. If you are on windows you can open cmd, type in "ipconfig" and find an adapter that has a "Default Gateway" shown. In your browser type that address in to the URL bar and you should be greeted with some type of login screen. If you don't know the login info ask anyone who might know for it before trying to reset. It might even be the default username and password combo which you can find out by searching for it on the web.

Here is where all I can say is either search online or look around for the options to do so, be careful messing around with settings.

So if you configured NAT for your VM or you have a dedicated machine for Debian, you will want to set a static IP for the computer that the VM/Debian is running on, you will need that computers MAC address.

If you are using Bridged mode for your VM you will have to use the VM's MAC address to set the IP.

2. Port forwarding

Navigate to your routers port forwarding interface. Make the inside and outside port TCP 8443 and set the local IP/host to the static IP you set beforehand. Leave anything regarding outside addresses, connection sources etc. blank.

Now your server can be reached through your routers outside address, through the 8443 port. If this is the router that your ISP connects to then you are good to go here. If you have another router(s) in front of yours then you need to create this port forwarding rule on every single one, of course the local address being not the server this time, but the routers outside address that you last configured port forwarding on.

At this point you can check if your entire port forward chain works via powershell on windows (Make sure Debian is connected and running). First find your public IP address, the easiest way is to open a site like <https://whatismyipaddress.com/> and copy what it shows.

Then open powershell and type in this command where IP is your public IP

```
tnc IP -Port 8443
```

If everything is good it should almost instantly say it succeeded

3. Dynamic DNS

So this is great for us because it gives us consistent access from outside into our local network. Instead of having to keep track somehow of our public IP address we just register, in our case to DuckDNS and remember that domain, which will always keep track of our public IP.

For this to work we need a device to be online that can run the DuckDNS software to keep track and report the public IP. This can be essentially any device thats behind your router, even the router itself if it supports it. If your router does (check around in settings or check online) I recommend using it, as the router is always online and essentially its a task that we balanced over to the router.

If you are running the VM, for example you can also use the host system for the DuckDNS software. But of course we can also use our Debian server, and thats what I will detail here.

3.1 Create your DuckDNS account by going to <https://www.duckdns.org/> and log in with your preferred method which you can see at the top of the site.

3.2 Then the page should show the option to create a domain. In the textbox simply type in the domain you'd like (eg. test-net, my-network, woodpigeon)

3.3 Now copy somewhere your token thats on top of the website and remember your domain, which will be DOMAIN.duckdns.org

3.4 Now we will follow the install guide for the linux-gui. On the top of the page click on installs. In the little dropdown menu select your domain, then click on "linux-gui"

First of all we install the dependencies

```
sudo apt-get install zenity cron curl
```

Then click on "linux GUI version" to download the shell script. Then move it to your home folder by running this command

```
mv ~/Downloads/duck-setup-gui.sh ~/
```

And then run these commands

```
chmod +x duck-setup-gui.sh  
./duck-setup-gui.sh
```

The setup will first ask for your domain and then the access token. And with this your domain is registered and will update its IP automatically! At this point you should be able to ping your domain and even run the tnc powershell command with your domain instead of the IP (:8443 still needed).

Configuring Matrix / Synapse

This setup follows the official guides

<https://element-hq.github.io/synapse/latest/setup/installation.html>

<https://element-hq.github.io/synapse/latest/postgres.html>

1. We add the official Matrix repo

```
sudo apt install -y lsb-release wget apt-transport-https
```

```
sudo wget -O /usr/share/keyrings/matrix-org-archive-keyring.gpg  
https://packages.matrix.org/debian/matrix-org-archive-keyring.gpg  
echo "deb [signed-by=/usr/share/keyrings/matrix-org-archive-keyring.gpg]  
https://packages.matrix.org/debian/ $(lsb_release -cs) main" | sudo tee  
/etc/apt/sources.list.d/matrix-org.list
```

```
sudo apt update
```

2. We install Synapse

```
sudo apt install matrix-synapse-py3
```

It will ask for a Server Name. This you cannot change later so make sure you enter it correctly. Here is where you enter your DuckDNS Domain, so example.duckdns.org

It will ask if you want to report statistics, you can choose either to your liking

2.1 We install necessary crypto libraries for encryption

```
sudo apt install libolm3 libolm-dev
```

3. Installing and configuring PostgreSQL

We cannot leave out PostgreSQL. By default Synapse uses SQLite but matrix-meta bridge requires PostgreSQL.

```
sudo apt install postgresql python3-psycopg2
```

We create a database user for synapse, the first command will get us into the sql prompt, the last one is how you exit out of it

```
sudo -u postgres psql
```

```
CREATE USER synapse WITH PASSWORD 'Password';
```

```
CREATE DATABASE synapse OWNER synapse LC_COLLATE 'C' LC_CTYPE 'C' TEMPLATE template0;
```

```
\q
```

Replace Password with an actual strong password. Make sure it doesn't include special characters like “\” or “?”.

4. Configure Synapse

We open the main configuration file

```
sudo nano /etc/matrix-synapse/homeserver.yaml
```

We link it to our newly created PostgreSQL database by finding and modifying the “database:” block so it looks like this:

```
database:
  name: psycopg2
  args:
    user: synapse
    password: Password
    database: synapse
    host: 127.0.0.1
    cp_min: 5
    cp_max: 10
```

Now we have to add a registration secret for which we will use a long random string. To generate one you need to exit nano (ctrl+x) and run this command

```
cat /dev/urandom | tr -dc 'a-zA-Z0-9' | fold -w 64 | head -n 1
```

Copy the outcome and go back into your config file and paste this line at the bottom

```
registration_shared_secret: "GENERATED_STRING"
```

Where GENERATED_STRING is the random string and we save the file (ctrl+s)

Here is what the final homeserver.yaml will look like, use it to make sure everything we needed so far is in here

```
# This is set in /etc/matrix-synapse/conf.d/server_name.yaml for Debian installations.
# server_name: "SERVERNAME"
pid_file: "/var/run/matrix-synapse.pid"
listeners:
  - bind_addresses:
    - ::1
    - 127.0.0.1
    port: 8008
    resources:
      - compress: false
        names:
          - client
          - federation
        tls: false
        type: http
        x_forwarded: true
database:
  name: psycopg2
  args:
    user: synapse
    password: [REDACTED]
    database: synapse
    host: 127.0.0.1
    cp_min: 5
    cp_max: 10
    #database: /var/lib/matrix-synapse/homeserver.db
log_config: "/etc/matrix-synapse/log.yaml"
media_store_path: /var/lib/matrix-synapse/media
signing_key_path: "/etc/matrix-synapse/homeserver.signing.key"
trusted_key_servers:
  - server_name: "matrix.org"
registration_shared_secret: "[REDACTED]"
public_baseurl: "[REDACTED]:8443"
app_service_config_files:
  - "/etc/matrix-synapse/mautrix-meta.yaml"
```

5. Starting Synapse

Lets start the service

```
sudo systemctl restart matrix-synapse
```

And we register the user that you will use to connect to your server through Windows Phone Matrix / Element

```
register_new_matrix_user -c /etc/matrix-synapse/homeserver.yaml
http://localhost:8008
```

Here it will ask for a new username and a password, make sure this one IS secure, type yes when asked if you should be an admin

6. Creating the SSL certs for the server to use + Nginx

This part is assuming you used DuckDNS, if you have local certs for local access (eg. openssl), or you have different certs you want to use, you can skip to the part where we tell Nginx the certs location.

Install Nginx and Certbot

```
sudo apt install nginx certbot
```

Make sure curl is installed

```
sudo apt install curl
```

Create the DuckDNS API Hook

```
sudo nano /etc/letsencrypt/duckdns-hook.sh
```

And paste this inside

```
#!/bin/bash
TOKEN="DUCKDNS_TOKEN"
DOMAIN="YOUR_DOMAIN"
curl -s "https://www.duckdns.org/update?
domains=$DOMAIN&token=$TOKEN&txt=$CERTBOT_VALIDATION"
sleep 30
```

Where DUCKDNS_TOKEN is your actual token from the duckdns website and YOUR_DOMAIN is your domain

Then save the file and make it executable

```
sudo chmod +x /etc/letsencrypt/duckdns-hook.sh
```

Get the SSL certs

```
sudo certbot certonly --manual --preferred-challenges dns --manual-auth-hook
/etc/letsencrypt/duckdns-hook.sh -d DOMAIN
```

Where DOMAIN is your domain

Now we open the nginx config file for Matrix

```
sudo nano /etc/nginx/sites-available/matrix
```

And paste this inside

```
server {
    listen 8443 ssl;
    server_name DOMAIN;

    ssl_certificate /etc/letsencrypt/live/DOMAIN/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/DOMAIN/privkey.pem;
```

```

location /_matrix {
    proxy_pass http://localhost:8008;
    proxy_set_header X-Forwarded-For $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Host $host;

    client_max_body_size 50M;
}

location /_synapse/client {
    proxy_pass http://localhost:8008;
    proxy_set_header X-Forwarded-For $remote_addr;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header Host $host;
    client_max_body_size 50M;
}
}

```

Where DOMAIN is your public domain. If you have local certs, or any other certs you want to use you just manually point it to their location. Here we declare that the port we will use to connect is 8443.

Save and exit the file.

By default Debian doesn't have an easily configurable firewall. Because this server will be open to the public it is best practice to install one.

```
sudo apt install ufw
```

Then allow port 8443 in UFW

```
sudo ufw allow 8443/tcp
```

Enable the site + restart Nginx

```
sudo ln -s /etc/nginx/sites-available/matrix /etc/nginx/sites-enabled/
```

```
sudo systemctl restart nginx
```

Then we add the public URL to Synapse

```
sudo nano /etc/matrix-synapse/homeserver.yaml
```

Add this line to the bottom of the file

```
public_baseurl: "https://DOMAIN:8443"
```

Then make sure in the "listeners" block in the port: 8008 section x_forwarded: true is set

```

listeners:
- port: 8008
  tls: false
  type: http
  x_forwarded: true

```

Now save this file and restart Synapse

```
sudo systemctl restart matrix-synapse
```

```
# This is set in /etc/matrix-synapse/conf.d/server_name.yaml for Debian installations.
# server_name: "SERVERNAME"
pid_file: "/var/run/matrix-synapse.pid"
listeners:
  - bind_addresses:
      - ::1
      - 127.0.0.1
    port: 8008
    resources:
      - compress: false
        names:
          - client
          - federation
        tls: false
        type: http
        x_forwarded: true
database:
  name: psycopg2
  args:
    user: synapse
    password: [REDACTED]
    database: synapse
    host: 127.0.0.1
    cp_min: 5
    cp_max: 10
    #database: /var/lib/matrix-synapse/homeserver.db
log_config: "/etc/matrix-synapse/log.yaml"
media_store_path: /var/lib/matrix-synapse/media
signing_key_path: "/etc/matrix-synapse/homeserver.signing.key"
trusted_key_servers:
  - server_name: "matrix.org"
registration_shared_secret: "[REDACTED]"
public_baseurl: "[REDACTED]:8443"
app_service_config_files:
  - "/etc/matrix-synapse/mautrix-meta.yaml"
```

At this point if you opted for plain public access you should be able to reach your server in a web browser on a public network (eg. mobile data) through this link

https://DOMAIN:8443/_matrix/client/versions

And if that works you should be able to download the Element app on your phone and connect to your server with the admin user you configured and the server set to: <https://DOMAIN:8443>

Congratulations on getting this far!

Configuring mautrix-meta bridge

0. First we create another database in PostgreSQL for our bridge

```
sudo -u postgres psql
```

```
CREATE USER mautrix_meta WITH PASSWORD 'BRIDGE_PASSWORD';
```

```
CREATE DATABASE mautrix_meta OWNER mautrix_meta;
```

```
\q
```

1. We make sure wget and ffmpeg is installed

```
sudo apt install -y wget ffmpeg
```

2. We get the bridge executable from the repo

We will get the executable directly from the repos latest release. If it says it cannot find the file in the repo you should check manually here: <https://github.com/mautrix/meta/releases>

If you are on an x64 CPU so most people, you need one thats meta-amd64.

At the time of writing this is these are the commands you would run:

```
mkdir -p ~/mautrix-meta
```

```
cd ~/mautrix-meta
```

```
wget https://github.com/mautrix/meta/releases/latest/download/mautrix-meta-amd64
```

```
chmod +x mautrix-meta-amd64
```

Now we run the binary so that it creates the default config file for us

```
./mautrix-meta-amd64 -generate
```

And it is time to edit the configuration

```
nano config.yaml
```

It is a huge file and especially in Nano its difficult to see through, so you will be using lots of ctrl+f

Find the “homeserver” section and make sure the address and domain are set

```
homeserver:
```

- address: http://localhost:8008
- domain: DOMAIN

```
# Homeserver details.
homeserver:
  # The address that this appserv
  # Local addresses without HTTP
  # but https also works if they
  address: http://localhost:8008
  # The domain of the homeserver
  domain: localhost.duckdns.org
```

Find the “database” section and make sure the uri is set

```
database:
  type: postgres
  uri: postgres://mautrix_meta:BRIDGE_PASSWORD@127.0.0.1:5432/mautrix_meta?sslmode=disable
```

```
database:
  # The database type. "sqlite3-fk-wal" and "postgres" are supported.
  type: postgres
  # The database URI.
  #   SQLite: A raw file path is supported, but `file:<path>?_txlock=immediate` is recommended.
  #   https://github.com/mattn/go-sqlite3#connection-string
  #   Postgres: Connection string. For example, postgres://user:password@host/database?sslmode=disable
  #   To connect via Unix socket, use something like postgres:///dbname?host=/var/run/postgresql
  uri: postgres://mautrix_meta:BRIDGE_PASSWORD@127.0.0.1:5432/mautrix_meta?sslmode=disable
  # Maximum number of connections.
  max_open_conns: 5
  max_idle_conns: 1
  # Maximum connection idle time and lifetime before they're closed. Disabled if null.
  # Parsed with https://pkg.go.dev/time#ParseDuration
  max_conn_idle_time: null
  max_conn_lifetime: null
```

Find the “bridge” section and make sure that inside of it permissions is set, username is the one you will use to login to the app

```
permissions:
  "DOMAIN": user
  "@username:DOMAIN": admin
```

```
# Permissions for using the bridge.
# Permitted values:
#   relay - Talk through the relaybot (if enabled)
#   commands - Access to use commands in the bridge
#   user - Access to use the bridge with permissions
#   admin - Full access, user level with permissions
# Permitted keys:
#   * - All Matrix users
#   domain - All users on that homeserver
#   mxid - Specific user
permissions:
  "*": relay
  "mautrix-duckdns.org": user
  "@elisa:mautrix-duckdns.org": admin

# Config for the bridge's database.
database:
```

Since we are only configuring this for messenger find the “network:” block and set it to

```
network:
  mode: messenger
```



```
network:
  # Which service is this bridge
  # * unset - allow users to pick
  # * facebook - connect to FB Me
  # * facebook-tor - connect to F
  # * messenger - connect to FB M
  # * messenger-lite - connect to
  # * instagram - connect to Inst
  #
  # Remember to change the appser
  mode: "messenger"
```

And make sure backfill is enabled

```
backfill:
  enabled: true
```

```
# Settings for backfilling messages.
# Note that the exact way settings are applied d
# See https://docs.mau.fi/bridges/general/backfi
backfill:
  # Whether to do backfilling at all.
  enabled: true
```

Now we need to generate a registration file

```
./mautrix-meta-amd64 -g
```

And we move this file to our Synapse config directory

```
sudo cp registration.yaml /etc/matrix-synapse/mautrix-meta.yaml
```

```
sudo chown matrix-synapse:matrix-synapse /etc/matrix-synapse/mautrix-meta.yaml
```

And now we edit the synapse config so that it knows to use it

```
sudo nano /etc/matrix-synapse/homeserver.yaml
```

Add this section at the bottom of your file

```
app_service_config_files:
  - "/etc/matrix-synapse/mautrix-meta.yaml"
```

```

# This is set in /etc/matrix-synapse/conf.d/server_name.yaml for Debian installations.
# server_name: "SERVERNAME"
pid_file: "/var/run/matrix-synapse.pid"
listeners:
  - bind_addresses:
    - ::1
    - 127.0.0.1
    port: 8008
    resources:
      - compress: false
        names:
          - client
          - federation
        tls: false
        type: http
        x_forwarded: true
database:
  name: psycopg2
  args:
    user: synapse
    password: [REDACTED]
    database: synapse
    host: 127.0.0.1
    cp_min: 5
    cp_max: 10
    #database: /var/lib/matrix-synapse/homeserver.db
log_config: "/etc/matrix-synapse/log.yaml"
media_store_path: /var/lib/matrix-synapse/media
signing_key_path: "/etc/matrix-synapse/homeserver.signing.key"
trusted_key_servers:
  - server_name: "matrix.org"
registration_shared_secret: "[REDACTED]"
public_baseurl: "[REDACTED]:8443"
app_service_config_files:
  - "/etc/matrix-synapse/mautrix-meta.yaml"

```

And restart synapse

```
sudo systemctl restart matrix-synapse
```

Then we start our bridge

```
./mautrix-meta-amd64
```

And lastly we create a systemd service to manage the bridge

```
sudo nano /etc/systemd/system/mautrix-meta.service
```

Inside paste this

```

[Unit]
Description=Mautrix-Meta bridge
After=network.target matrix-synapse.service

[Service]
User=root
WorkingDirectory=/root/mautrix-meta
ExecStart=/root/mautrix-meta/mautrix-meta-amd64
Restart=always

```

```
[Install]  
WantedBy=multi-user.target
```

And we enable to start it

```
sudo systemctl daemon-reload
```

```
sudo systemctl enable mautrix-meta
```

```
sudo systemctl start mautrix-meta
```

And if everything is configured correctly we are in the final stretch, the only thing left is telling the bot our login token and we are good to go!

The bridge bot

First you will want to download the Element app on your phone. I have one running Android so I got the Element classic app from the playstore.

Once installed you open it and login with the username you configured, the password you set and the server address being <https://DOMAIN:8443>

Once logged in you press the green button on the bottom right and select start chat and in the search bar type: @metabot:DOMAIN

The bot user should appear and select it to start a new chat room. If you send the message “ping” it should reply something, verifying that its up and running.

I recommend doing this on a desktop. You want to open <https://messenger.com> and make sure you are logged in. Press F12 to open DevTools and open the Network tab.

Refresh the website and at the very top of the DevTools network box there should be an item called “messenger.com”

Select it, right click it and select “copy as cURL”

Now you need to get this over to your phone, I chose the old fashioned email solution. Send an email to yourself by just pasting the contents of your clipboard to the message body and thats it.

Back on your phone open your email client and copy the entire content you pasted. (and delete the email, message afterwards...)

Back in the chat, send “login” and the bot will reply.

Now is when you send the cURL you copied from your email.

And done! If everything is configured correctly you should start getting a bunch of invites that you can accept!

Logging in to Windows Phone Matrix

The very last step, first install the Windows Phone Matrix certificate, dependencies and then the appx from my repo.

<https://github.com/ElisaC04/WindowsPhoneMatrix/releases>

Open the app and just as you did in Element login. Difference being here for the server field you only need to type DOMAIN:8443

Here you will also see a field called metaID, this is not required. Currently the way the app works is by default everyone has their metaID as their name. The only difference being you because you will appear as “Me” when you open a chat.

The way I solved it for now is that other participants username is the same as the chat rooms name, which for 1-1 chats is always the other persons name. The only flaw this has is in group chats everyones name is the group chat name.

And if you send a message from any official messenger app, facebook website you will see those messages as sent by your facebook profile, not “you” (synapse user).

To get around this go back into the Element app, open a chat and find a message sent by your facebook user and click on the profile picture. There you will see its metaID, copy just the numbers noting more or less. And this is what you paste in the metaID field, if done properly you will see every message sent by your facebook user as actually sent by you!